

Trabalho 2º Trimestre – Campo Minado

Instituto Federal de Educação, Ciência e Tecnologia Catarinense –
Campus Concórdia

Disciplina: Programação I
Docente: Alisson Borges Zanetti

Discente: Lauren Manica Conceição
Turma: 1F

1. Introdução:

Com base nos requisitos da avaliação do 2º trimestre da disciplina de Programação I, foi desenvolvido um programa na linguagem Python com o objetivo de desenvolver o jogo Campo Minado, aplicando os conceitos estudados durante as aulas, como matrizes, estruturas de repetição, listas e funções.

Os requisitos da avaliação incluíam a criação de um tabuleiro representado por matrizes, a distribuição aleatória de minas, a contagem de minas adjacentes, as condições de vitória e derrota e a criação de funcionalidades extras.

Este relatório apresenta os passos seguidos para o desenvolvimento do programa, as fontes consultadas, a explicação da lógica utilizada nas principais funcionalidades, os problemas encontrados durante a implementação e as soluções adotadas para resolvê-los (diretrizes retiradas das orientações).

2. Desenvolvimento:

2.1. Sobre os passos tomados:

O sistema foi montado com os seguintes processos:

- I. Criação de menu (escolha entre Modo Individual ou Multiplayer);
- II. Criação do tabuleiro;
- III. Criação das bombas;
- IV. Criação e armazenamento dos palpites;
- V. Função de demarcação de bombas adjacentes;
- VI. Criação do Modo Multiplayer.

Inicialmente, foi estudado o modelo do tabuleiro e como os palpites e coordenadas seriam determinados, tomando como referência os trabalhos desenvolvidos por Matheus Delduque (2022) e Bernardo Gozdecki (2026).

Como base, foi desenvolvido o menu, o qual dá as opções de jogo Individual ou Multiplayer, sendo estes, respectivamente:

1. **Individual:** No Modo Individual, a posição das bombas é calculada automaticamente por uma futura função **random** e armazenada em uma lista.
2. **Multiplayer:** No Modo Multiplayer, a posição das bombas é definida por um dos integrantes da dupla e armazenada em uma lista.

Na gamificação do sistema, foi criado um tabuleiro no formato 6×7, posteriormente alterado para 7×7, os inputs para o palpite do usuário (**coluna (inteiros) e fileiras (strings)**) e a função “**PosiçãoDeBombas()**”, a qual possui o intuito de sortear onde a bomba estaria com a junção de um inteiro e uma string, utilizando o pacote **random** e suas funções **random.randint()** e **random.range()** e o pacote **string** e sua função **string.ascii_uppercase**.

Por fim, a função “**PosiçãoDeBombas()**” é transformada em uma lista, armazenando a coordenada de cada bomba e retornando e atribuindo a lista para a variável **Bomba**, a qual será usada como uma variável global e também para o uso do modo Multiplayer. Vale lembrar que as funções **random** foram todas criadas em intervalos de 1 a 7, tanto nos números quanto as letras, devido o tamanho do tabuleiro.

Posteriormente, foi elaborado uma seguinte função chamada “**PosiçãoDoJogador()**”, que junta a coluna (inteiro) com a fileira (string), transformando-a em uma coordenada (string), assim como as bombas. Assim, podemos efetuar uma comparação para averiguar se a casa sugerida pelo usuário não é uma bomba (caso contrário, ele perdeu), com a sintaxe “**if Posição_do_Jogador in Bombas**”.

Se este questionamento for verdadeiro, o jogo irá imprimir “**Casa com bomba. Você perdeu!**”. Caso contrário, o usuário poderá prosseguir o jogo, utilizando a função **continue**.

Após isso, criei a função **Marcação()**, que capta a coordenada determinada pelo usuário, e caso não seja uma bomba, analisa se não há bombas em seu entorno (bombas vizinhas ou bombas adjacentes) com o auxílio de diversas variáveis. Em seguida, com base em um contador em loop no sistema de True ou False, substitui o símbolo da casa pela quantidade de bombas ao redor da casa escolhida.

Esta foi uma das funções mais complexas de realizar, pois foi preciso utilizar de manipulações de strings ainda não conhecidas, como a **ord()**, que possui o intuito de calcular a posição de cada caractere na Tabela ASCII.

Neste ponto, o tabuleiro já havia sido formado, condições do menu já haviam sido feitas, as bombas eram criadas automaticamente e a marcação estava funcionando corretamente.

Por fim, como bônus, iniciei o Modo Multiplayer. Este processo em si não possuiu tanta complexidade em sua criação, pois reutilizei o código do modo individual, adicionando apenas novos inputs para a digitação da lista por um dos integrantes da dupla, e criei uma função chamada **BombasMultiplayer()**, onde os inputs (colunas e fileiras) também são armazenados em listas, e a lista atribuída novamente para a variável global **Bombas** funcionando assim, com as outras mesmas funções do Modo Individual.

2.2. Sobre os problemas e suas soluções:

- I. **Matrizes:** O problema inicial foi a criação de um tabuleiro em matriz, pois inicialmente, devido a desatenção nas solicitações do trabalho, estava criando o tabuleiro em **strings**, pelo método **f-string**. Por fim, ao reler as orientações, refiz o tabuleiro 7x7 em matrizes.
- II. **Bombas aleatórias:** A criação de bombas e a manipulação das funções **random.randint()** e **random.range()** não foram complicadas, sendo preciso apenas determinar o intervalo de números. O problema em si era a repetição de números, strings ou coordenadas já existentes na lista. Assim, foi necessário consultar a plataforma StackOverflow e me baseei na resposta dada. Com isso, a função das bombas passou a desenvolver posições não repetidas.
- III. **Marcações e bombas adjacentes:** Em questão às marcações e as bombas adjacentes, foi um dos processos mais complexos e difíceis de realizar, e precisei consultar a Inteligência Artificial ChatGPT para encontrar uma função que pudesse se encaixar em minha lógica, tendo em análogo a função **ord()**. Por fim desta função, na etapa de localização das bombas adjacentes e contagem destas, pude realizar sem dificuldade, utilizando do retorno de True ou False para a contagem.

Por fim, nas etapas de criação Modo Multiplayer, desenvolvimento da função **Posição_do_Jogador()** e o desenvolvimento da função **BombasMultiplayer()** foram significativamente fáceis e tão pouco exigentes.

Referências:

William J. Turkel e Adam Crymble, Manipular Strings com Python, **Programming Historian**, 2016 <https://programminghistorian.org/pt/licoes/manipular-strings-python>. Acesso em: 24 jun. 2026.

MANIERO, [TypeError: not all arguments converted during string formatting \(Python 3.4\)](https://pt.stackoverflow.com/questions/51574/typeerror-not-all-arguments-converted-during-string-formatting-python-3-4), **StackOverflow**, 2016 <https://pt.stackoverflow.com/questions/51574/typeerror-not-all-arguments-converted-during-string-formatting-python-3-4>. Acesso em: 25 jun. 2026.

DELDUQUE, Matheus, Campo Minado em Python, **Youtube**, 2022. Disponível em: <https://youtu.be/7NTYE2rIDxI?si=SeNN9U8u1XtxiZ98>. Acesso em: 26 jun. 2026.

Autor desconhecido, Como remover os colchetes e as vírgulas em uma lista no Python?, **StackOverflow**, 2020. Disponível em: <https://pt.stackoverflow.com/questions/481177/como-remover-os-colchetes-e-virgulas-ao-imprimir-uma-lista-em-python>. Acesso em: 26 jun. 2026.

WOSS, Gerar números aleatórios em Python sem repetir, **StackOverflow**, 2021. Disponível em: <https://pt.stackoverflow.com/questions/195422/gerar-n%C3%BAmmeros-aleat%C3%B3rios-em-python-sem-repetir>. Acesso em: 10 jul. 2026.

WOSS, [Identificar elementos repetidos em lista com Python](https://pt.stackoverflow.com/questions/216413/identificar-elementos-repetidos-em-lista-com-python), **StackOverflow**, 2023 <https://pt.stackoverflow.com/questions/216413/identificar-elementos-repetidos-em-lista-com-python>. Acesso em: 10 jul. 2026.

GOZDECKI, Bernardo, Campo Minado em Python!, **Youtube**, 2026. Disponível em: <https://youtu.be/rRE03OfUXaM?si=y609EXD7TToDFypZ>. Acesso em: 22 jul. 2026.

Autor desconhecido, Método replace() de string em Python, **W3 Schools**, Data desconhecida. Disponível em: https://www.w3schools.com/Python/ref_string_replace.asp. Acesso em: 22 jul. 2026.

WHARDANI, Mamta, Python ord(), **Code Academy**, 2025. Disponível em: <https://www.codecademy.com/resources/docs/python/built-in-functions/ord>. Acesso em: 22 jul. 2026.

Fora utilizado para consulta e revisão do código (sem usurpar dos códigos demonstrativos) a Inteligência Artificial ChatGPT.